

PixelPacker: Background-Aware Neural Image Compression for Wildlife Edge Systems

Anonymous Authors

Abstract—We present PixelPacker, an end-to-end neural image compression framework for low-bandwidth, energy-constrained wildlife monitoring systems. Unlike traditional codecs, which degrade severely at low bitrates, or recent neural approaches, which are too computationally intensive for remote camera traps, PixelPacker adopts a decoupled edge–cloud design that achieves extreme compression while delivering better perceptual quality than traditional image compression schemes and comparable to neural codecs. At the edge, a MiniEncoder obtained via knowledge distillation and quantization-aware training runs on a dedicated FPGA, achieving a 36× smaller footprint than the teacher model. By exploiting the largely static nature of camera-trap imagery, the system transmits only sparse foreground differences relative to a learned background representation, enabling adaptive bitrate control at runtime. On the cloud side, a fine-tuned decoder restores fine details to maintain high perceptual quality. Evaluations demonstrate up to 200× compression over raw images and >10× lower bitrate than JPEG’s maximum compression, while sustaining comparable perceptual index scores and achieving energy efficiency exceeding 66 GOPS/J. Designed for camera traps with LoRaWAN links, PixelPacker achieves application-critical latency and energy reductions, extending deployment lifetimes on limited batteries. By uniting algorithmic innovations with hardware-aware design, PixelPacker sets a new benchmark for scalable, real-time wildlife monitoring and generalizes to other resource-constrained edge intelligence scenarios.

I. INTRODUCTION

Camera traps have become indispensable in wildlife conservation and ecological research, enabling critical applications such as species identification, population estimation, behavioral analysis, and real-time anti-poaching alerts [1], [2], [3], [4]. Typically deployed in remote locations lacking reliable internet connectivity, these battery-operated devices can only operate on low-bandwidth, intermittent, and costly communication links for data transmission [5], [6], [7], [8], [9]. Due to these constraints, manual retrieval of images remains the predominant approach, significantly hindering timely data access and increasing operational costs [10]. This latency can delay critical conservation decisions by weeks or even months, undermining the potential for rapid intervention. Thus, researchers and conservationists have consistently identified the lack of timely remote visual access as a primary bottleneck, highlighting the insufficiency of transmitting only low-bandwidth inference alerts (e.g., event notifications), since verifying species identity, interpreting complex behaviors (such as foraging or conflicts), or confirming poaching incidents often requires full contextual imagery [11], [12], [13], [14], [15].

Transmitting data over extremely low-bitrate channels without losing critical visual details poses a fundamental challenge

for edge-based wildlife monitoring. While video capture could provide richer temporal context, continuous recording is often infeasible in the field due to severe power and storage constraints, accelerated battery depletion, and prolonged encoding times that risk missing subsequent events [16]. Consequently, still images triggered by motion or sensor events remain the most practical solution [17]. This necessitates an image compression framework capable of operating under ultra-low-bitrate, intermittent links typical of remote deployments (e.g., LoRaWAN [18] or satellite backhaul [19]), while maintaining high visual fidelity and low latency. Traditional codecs (i.e., JPEG [20]) degrade image quality substantially at low bitrates, making subsequent analysis difficult or impossible. Recent neural image compression approaches offer substantial improvements at these bitrates by utilizing learned, content-aware representations [21]; however, their computational complexity, large memory requirements, and power consumption render them unsuitable for battery-constrained edge deployments typical of wildlife camera traps.

This challenge reveals a fundamental tension in energy-constrained edge systems: aggressive neural compression reduces the payload size, minimizing the time for high-energy cost radio transmission, but incurs a significant energy cost, which makes it infeasible for edge computation. Conversely, simpler codecs have low computational energy but produce larger payloads that demand excessive transmission energy and time. This tradeoff becomes even more pronounced in latency-critical applications.

The key to resolving this trade-off lies in leveraging environment-specific characteristics. For wildlife monitoring, the static positioning of camera traps presents a unique and powerful opportunity for optimization that is otherwise unavailable in more general scenarios. This fixed viewpoint means that the background in the imagery remains largely constant across multiple captures. This perspective reframes the problem, leading to a more nuanced research question:

How can the fixed positioning of wildlife camera traps be leveraged to design an extreme image compression system that jointly maximizes compression and image quality while minimizing energy consumption, and transmission cost, through an optimized computation–communication tradeoff?

To address this question, we propose PixelPacker (Fig. 1), an edge–cloud split, end-to-end adaptive bitrate neural image compression framework that rests on two key innovations: a novel hardware-friendly compression scheme that maximally exploits static backgrounds and a specialized, hardware-

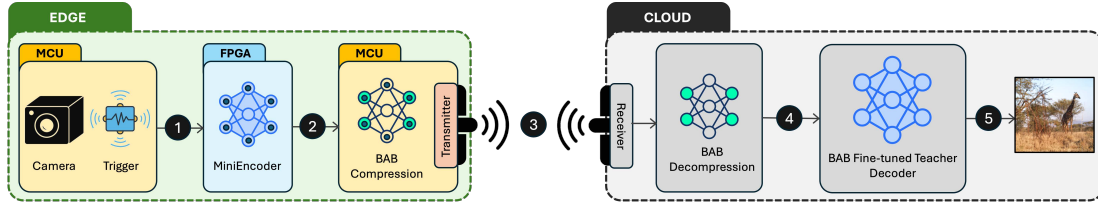


Figure 1: PixelPacker’s System Pipeline. Upon motion detection, the MCU triggers the FPGA’s MiniEncoder. The resulting latent is compressed by the BAB module for transmission. In the cloud, the bitstream is decompressed, and a fine-tuned decoder reconstructs a high-fidelity image from the latent.

accelerated encoder designed for energy efficiency. To our knowledge, PixelPacker is the first system to integrate these solutions. Our key contributions are:

- [C1] An Asymmetric Edge–Cloud Architecture for High-Fidelity Reconstruction:** We design a hardware-efficient encoder through distillation and quantization for deployment on the edge, paired with a full-capacity decoder operating in the cloud. The decoder is fine-tuned to the characteristics of our compression pipeline, enabling accurate recovery of fine-grained details and ensuring reconstructions meet the perceptual and structural fidelity requirements of real-time wildlife monitoring.
- [C2] A Background-Aware Bottleneck (BAB) for Extreme Compression:** We introduce a novel, *hardware-friendly* latent compression scheme that leverages the static nature of camera trap scenes. By maintaining a background codebook and transmitting only sparse, low-entropy latent differences, the BAB is able to achieve extreme compression rates.
- [C3] An Energy-Efficient FPGA Accelerator for Real-World Deployment:** Recognizing that general-purpose processors cannot meet the operational demands, we implemented our neural encoder as a specialized accelerator on a low-cost Xilinx Artix-7 FPGA. Our hardware-centric approach embeds model weights into dedicated parallel pipelines to eliminate memory bottlenecks. This delivers the low latency and minimal power consumption required for practical, long-term, battery-powered deployments.

Our comprehensive evaluation on a subset of the Serengeti Dataset [22], a wildlife, domain-specific collection of 2.65M camera trap sequences, highlights that PixelPacker successfully balances performance across demands of compression ratio, image quality, and energy efficiency. Our system achieves compression rates of up to 200× over raw imagery and over 10× lower bitrates than JPEG’s maximum compression setting, while maintaining image quality. To achieve a comparable Perceptual Index (an Image Quality Assessment metric), JPEG requires file sizes that are nearly 20× larger. Importantly, this performance is achieved with outstanding efficiency, exceeding 66 GOPS/J in energy utilization.

By successfully optimizing energy consumption, latency, and deployment cost, PixelPacker establishes a benchmark for real-time, intelligent wildlife monitoring systems, with design choices that readily generalize to other resource-constrained sensing and edge intelligence applications.

II. BACKGROUND AND RELEVANT WORKS

PixelPacker draws and builds on advancements in (a) image compression, (b) edge machine learning, and (c) edge neural image compression, while being specifically tailored to the unique (d) requirements of wildlife imagery.

A. Image Compression

Traditional codecs such as JPEG, and WebP have long served as the standard for lossy image compression [20], [23]. These hand-engineered methods rely on fixed transforms and manually designed quantization tables to reduce storage requirements. However, at ultra low bitrates, image quality deteriorates sharply making these codecs unsuitable for low bitrate applications. To address these limitations, Neural Image Compression (NIC) frameworks have emerged. Unlike traditional codecs, NIC methods optimize Shannon’s rate–distortion trade-off directly by learning data-driven transforms and probabilistic models in an unsupervised setup. The rate corresponds to the entropy of the latent representation and the distortion measures reconstruction error [24].

In general, neural image compression (NIC) frameworks comprise of an encoder that maps the input image to a compact latent representation, a bottleneck that quantizes this latent into discrete symbols for entropy coding, and a decoder that reconstructs the image from the quantized representation. Variational Autoencoders (VAEs) and Generative Adversarial Networks (GANs) have been the dominant architectures employed in NIC [25], [26]. VAEs transform the input into a probabilistic latent distribution, but the structured latent space often leads to overly smooth or blurry reconstructions [27]. In contrast, GAN-based methods employ a generator–discriminator framework, achieving reconstructions with high perceptual fidelity at very low bits-per-pixel (bpp), albeit at the expense of metrics such as PSNR[28].

Most modern models utilize hyperprior networks to achieve extremely low bit-rates. These provide side information in the form of hyperlatents, which describe the distribution of the primary latents and enable more accurate probability estimates for entropy coding [29]. Hyperprior-based models, combined with VAE or GAN-based NIC architectures, have allowed neural methods to significantly surpass traditional codecs in rate–distortion performance [30]. More recently, diffusion-based NIC approaches have been proposed, achieving extreme compression rates through generative reconstructions. However, these models often suffer from artifacts and hallucinated details [31].

Although NIC methods exhibit strong performance in optimizing the rate–distortion trade-off, they typically involve millions of parameters, leading to substantial computational and memory demands. As a result, most existing NIC architectures are unsuitable for deployment in resource-constrained edge computing scenarios.

B. Edge Machine Learning

Several techniques have been developed to adapt neural networks for resource-constrained settings. These approaches revolve around lowering the memory footprint or inference latency through reducing model size and making hardware friendly modifications to operations. Among such approaches, knowledge distillation [32] stands out as an effective method in which a large teacher network transfers knowledge to a smaller student model, enabling comparable performance with substantially fewer parameters. This technique is used in junction with quantization, which reduces model size and accelerates inference by representing weights and activations with lower-precision formats, often with minimal loss in accuracy [33]. Quantization can be applied post-training (PTQ) or incorporated during training through QAT [34], which introduces noise to simulate the effect of quantization and typically achieves better accuracy than PTQ.

Structural optimizations further improve efficiency by re-designing network operations. Depthwise and pointwise convolutions decompose standard convolutions into smaller, more efficient operations: depthwise convolutions apply a single filter per input channel, while pointwise convolutions (1×1 kernels) combine these outputs across channels. This factorization drastically reduces computation while preserving representational power [35].

Together, these methods form the backbone of modern approaches to model compression, making it feasible to deploy high-performing neural networks on edge hardware and embedded systems.

C. Edge Neural Image Compression

These techniques have been used in edge neural image compression, which aims to bring the benefits of learned compression to highly resource-constrained environments such as microcontrollers (MCUs) and FPGAs. Among such existing work is MCUCoder, an adaptive bitrate video compression model that employs an asymmetric encoder–decoder architecture tailored for MCUs [36]. It achieves a lightweight design with energy consumption comparable to Motion JPEG (M-JPEG) while delivering lower bitrates. Following this line of work, EASZ has been proposed as a transformer-based framework for IoT and edge scenarios, using an erase-and-squeeze process to reduce edge computation and offload reconstruction to the server side [37]. Another line of work introduces a hardware-aware learned image compression (LIC) framework that integrates knowledge distillation and a hardware-friendly implementation of the Generalized Divisive Normalization (GDN) activation [38].

Despite their promise, existing edge-focused NIC approaches face important limitations. MCUCoder and EASZ, while efficient on MCUs, are not able to achieve extreme compression rates, which is required in edge settings for transmission. The FPGA-based LIC framework is impractical for deployment, as it offers limited compression gains and relies on high-end, power-hungry FPGAs that are incompatible with IoT-scale energy constraints. Furthermore, none of these models exploit background redundancy, resulting in unnecessary bitrate expenditure on redundant features and leaving them suboptimal for bandwidth- and latency-critical wildlife monitoring.

D. Requirements of Wildlife Imagery

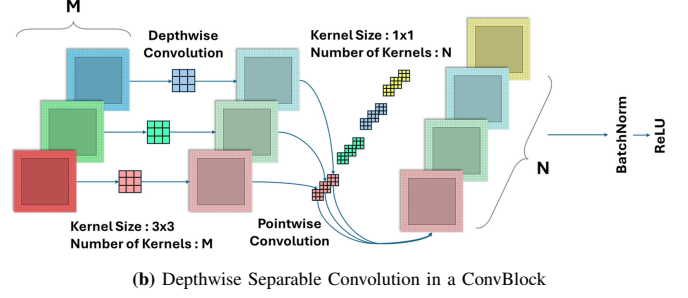
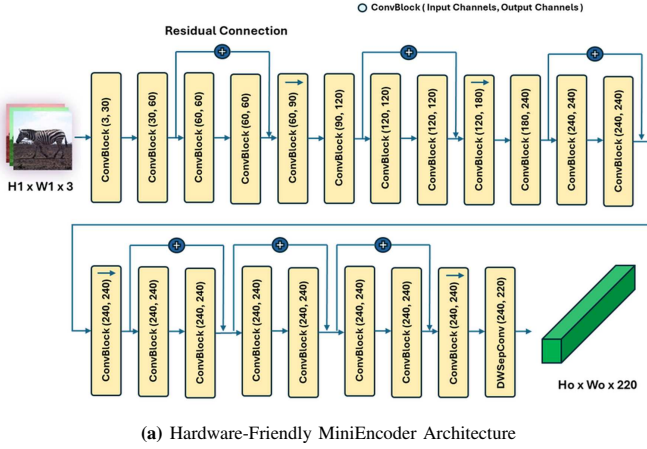
In wildlife monitoring applications, image quality requirements differ across stakeholders, however they all emphasize the need to preserve clear morphological landmarks in compressed images for reliable ecological analysis [39]. Compression methods must therefore minimize artifacts that increase measurement uncertainty, particularly at object boundaries, image peripheries, and under challenging conditions such as dynamic illumination. Models based on diffusion, while capable of extreme compression, are unsuitable in this context due to their tendency to introduce artifacts and hallucinated content in reconstructed images. Effective compression schemes must instead preserve fine edge definitions, maintain mid-level convolutional features essential for downstream detection tasks, and support real-time inference speeds [40]. Among existing approaches, GAN-based NIC models are the most promising for wildlife imagery, as they prioritize perceptual realism and visual fidelity at low bitrates.

III. DESIGN GOALS AND DECISIONS

A. Design Goals

Camera traps in remote wildlife environments operate under strict power and bandwidth constraints, requiring long-term autonomous operation. An edge image compression system must therefore balance energy efficiency, compression ratio, and reconstruction quality, while exploiting the static nature of camera traps to enable more aggressive compression for transmission. Guided by these considerations, our system is designed around the following goals:

- [G1] Enable Feasible Transmission over Ultra-Low-Bandwidth Networks:** The system must achieve compression ratios extreme enough to make the transmission of high-resolution images practical over costly, intermittent, and low-bandwidth channels (e.g., LoRaWAN, satellite).
- [G2] Ensure High Perceptual Fidelity for Downstream Analysis:** The reconstructed images must be of high perceptual quality and preserve critical visual information necessary for real-world downstream analysis. This means maintaining the integrity of morphological details, textures, and object boundaries required by ecologists for species identification and behavioral studies.



(a) Hardware-Friendly MiniEncoder Architecture

Figure 2: (a) Hardware-Friendly MiniEncoder architecture composed entirely of convolutional blocks and residual connections. The arrows inside the ConvBlocks indicate downsampling of the output dimensions, implemented via a stride of 2. (b) Detailed architecture of a single ConvBlock, which begins with a depthwise convolution followed by a pointwise convolution across channels. The resulting depthwise separable convolution output is then passed through Batch Normalization and a ReLU activation.

[G3] Minimize Total Energy Consumption within a Constrained Power Budget: For long-term, unattended deployments, the system’s total energy draw, encompassing both on-device processing and radio transmission, must be minimized. The design must navigate the inherent tradeoff between the energy spent on computation and the energy spent on communication to maximize operational lifetime on a limited battery.

These goals form the basis of our evaluation, ensuring that performance is measured not only in terms of compression ratio but also in terms of practical utility for real-world wildlife monitoring deployments.

B. Design Decisions

1) Asymmetric Encoder Design via Knowledge Distillation:

To design a highly compact encoder that retains image quality, we distill features from the Implicit Local Likelihood Model (MS-ILLM) [41], a state-of-the-art neural compression model. MS-ILLM’s GAN-based training enables reconstructions with near-realistic fidelity, which allows it to transfer rich feature information during distillation. This ensures that the student retains visual fidelity [G2] with far fewer parameters and lower inference times [G1]. Furthermore, MS-ILLM’s encoder uses convolutions and Rectified Linear Unit (ReLU) activations, which are far more hardware-friendly and lightweight in energy footprint for the edge deployed student encoder, compared to self-attention and other sophisticated methods [G3].

2) Hardware-Aware Optimization with Quantization-Aware Training: We use an FPGA as our co-processor because it delivers computation in short, parallel bursts, leading to higher instantaneous, but lower overall power draw and an extended operational lifetime compared to other general-purpose accelerators. [G3]. To maximize the efficiency of the FPGA deployment, we adopt QAT, which lowers weights and activations from 32-bit floating-point to 8-bit integer precision.

This results in a 4 times smaller memory footprint, ideal for limited on-chip storage. Furthermore, integer arithmetic consumes fewer DSP resources than floating-point operations, lowering inference latency [G1]. This comes at the cost of only a subtle quality drop [34], thus preserving perceptual fidelity for downstream analysis [G2].

3) Exploiting Background Redundancy with the BAB:

We introduce BAB to aggressively reduce the transmitted payload size. Rather than repeatedly sending near-identical background content, BAB maintains a representative latent codebook and transmits only the foreground variations relative to a static background reference, resulting in a more optimal balance among compression efficiency, energy consumption, and reconstruction fidelity. BAB achieves this balance by significantly reducing payload size, since only a compact difference (Δ) is transmitted rather than the full latent [G1]. Δ exhibits lower entropy, which allows compression with a lightweight entropy coder like zlib [42] instead of prohibitively large deep learning models [G3]. BAB also enables adaptive bitrate through a post-processing scaling factor, avoiding the need to train and store multiple models. This allows runtime adjustment of quality and compression with a single encoder, providing flexibility under varying bandwidth and application quality requirements with no extra energy cost.[G2] This algorithm is further explained in Section IV of the paper under the Background-Aware Bottleneck heading.

4) Cloud-Side Reconstruction with BAB Fine-Tuned Decoder:

Due to our decoupled architecture, decompression is performed in the cloud. This allows us to adopt an asymmetric design, which utilizes a heavier, more powerful decoder to ensure high-quality reconstructions. To mitigate quality loss introduced by the BAB, we modify the pre-trained decoder to restore fine details in regions of interest. This asymmetric design preserves perceptual fidelity comparable to state-of-the-art codecs [G2].

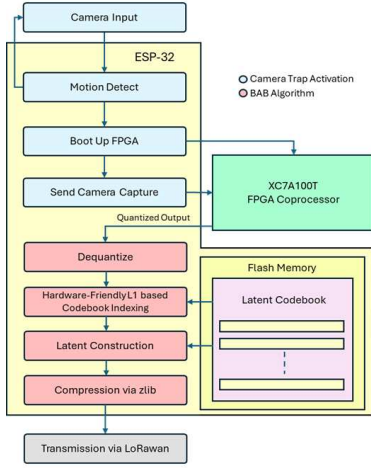


Figure 3: Detailed edge-side processing flow of PixelPacker. The ESP-32 handles motion detection, FPGA boot-up, and transmission, while the FPGA coprocessor performs encoding. The ESP-32 then executes BAB operations on the latent before transmitting the compressed.

IV. PIXELPACKER: SYSTEM DESIGN AND IMPLEMENTATION

Our system is architected as a decoupled edge-cloud solution. The process begins on the camera trap, where a motion detection event on the microcontroller triggers an image capture. This image is streamed from the MCU’s SRAM to the FPGA co-processor, initiating the edge-side compression pipeline. The goal is to generate a highly compressed latent representation for efficient transmission. This compressed data is then sent to the cloud for full reconstruction. The general overview of the data flow is highlighted in Fig. 1.

A. MiniEncoder $\rightarrow$ Edge

The first step in our edge pipeline is feature extraction, handled by the MiniEncoder. It is a lightweight neural network designed from the ground up for efficient hardware implementation. To create this compact yet powerful encoder, we utilize a new architecture and a specialized training process.

Model Design and Training: Our MiniEncoder is derived from a larger 7.1M FP32 parameter teacher encoder, a 5-layer CNN with ReLU activation and channel normalization, but is completely redesigned for edge efficiency (Fig. 2a).

The MiniEncoder makes use of depthwise separable convolutions instead of standard convolutions to reduce the memory footprint and accelerate the inference of the model by reducing parameter count and number of operations. It deliberately adopts a deeper architecture than the teacher with more convolutional blocks but smaller channel dimensions. This design minimizes peak activation memory, allowing the model to operate under tight memory budgets. Due to the deep architecture, we introduce residual connections for better gradient flow. We restrict all our residual blocks to 240 channels, corresponding exactly to the 240 DSPs on the Artix-7 100T FPGA, ensuring maximal efficiency and utilization. The MiniEncoder only uses 3×3 kernels for convolutions, avoiding the need to design conditional logic for different

kernel shapes, easing bandwidth and DSP utilization on the FPGA and achieving better timing closure by decreasing routing congestion. This modification also allows for hardware blocks to be tiled or reused without additional specialization as each convolutional layer is architecturally identical[43][44]. Furthermore, to introduce nonlinearity, we stick to ReLU activations instead of the generalized divisive normalization (GDN) commonly used in NIC. While GDN is effective for entropy control, ReLU is computationally inexpensive on hardware and can fuse directly into the weights during QAT. Similarly, Batch Normalization (BN) is chosen over the teacher’s Channel Normalization (CN) because BN parameters can be pre-computed and fused with convolutional weights during quantization, incurring no runtime overhead unlike CN. This redesign reduces the parameter count to 800K, an $9\times$ reduction from the teacher.

Knowledge Distillation: To train this redesigned architecture, we employ a hint-based knowledge distillation strategy. We train the MiniEncoder on the OpenImages V7 dataset [45], which consists of 1.7 million training images. We use Open-Images because its diversity helps the model capture more generalizable patterns, ensuring robustness across various applications. The objective combines a distillation term $\mathcal{L}_{\text{distill}}$ and a reconstruction term $\mathcal{L}_{\text{recon}}$. The distillation loss combines magnitude and directional alignment between teacher and MiniEncoder feature maps. $\mathcal{L}_{\text{magnitude}}$ is enforced via mean squared error between feature maps and $\mathcal{L}_{\text{direction}}$ is achieved by pooling feature maps into channel vectors and computing their cosine similarity. For each guided layer l , we take a weighted sum of the two terms, and the final loss is a weighted aggregation of these contributions across all selected layers (Eq. (1)). We select the first, middle and final layer of MS-ILLM for distillation loss.

$$\mathcal{L}_{\text{distill}} = \sum_{l=1}^3 \beta^l (\mathcal{L}_{\text{magnitude}}^l + \alpha^l \mathcal{L}_{\text{direction}}^l) \quad (1)$$

For reconstruction loss $\mathcal{L}_{\text{recon}}$, the MiniEncoder’s latent is passed through the frozen teacher decoder to reconstruct images. Similarity with the original image is evaluated using multi-scale structural similarity (MS-SSIM) for structural fidelity and learned perceptual image patch similarity (LPIPS) for perceptual alignment.

The final training objective is:

$$\mathcal{L}_{\text{total}} = \beta \mathcal{L}_{\text{distill}} + \gamma_1 \mathcal{L}_{\text{MS-SSIM}} + \gamma_2 \mathcal{L}_{\text{LPIPS}} \quad (2)$$

Before full training, we perform Bayesian optimization on a smaller subset of images to tune hyper-parameters β , γ_1 , and γ_2 , along with learning rate and scheduler configurations. The resulting values are then used for full-scale training.

Quantization-Aware Fine-Tuning: Following distillation, we perform QAT [34] to prepare the model for efficient integer-only arithmetic on the same dataset and loss functions. During this phase, the model also learns INT8 precision weights from

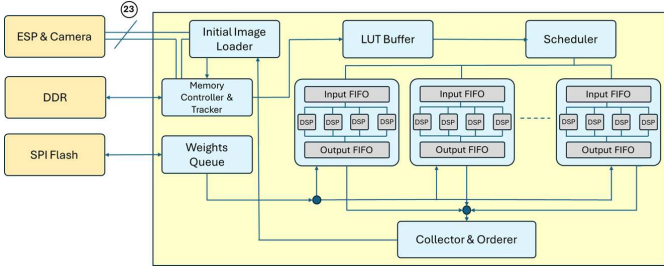


Figure 4: FPGA Architecture. The FPGA coprocessor encodes images on-device, with the memory controller managing DDR and SPI Flash access. Data passes through the LUT buffer, scheduler, and pipelined DSP modules, and the collector aggregates outputs for transmission to the ESP.

the original FP32 representation, yielding a $4\times$ reduction in model size and enabling more efficient arithmetic on FPGAs compared to floating-point operations. In the end, the student encoder only has a memory footprint of 800 kB compared to the teacher’s footprint of 28.4 MB offering a $36\times$ reduction. *FPGA Implementation:* With the model fully trained and quantized, it is deployed on the Artix-7 FPGA co-processor. The FPGA and the MCU interface via a 23 bit parallel bus, allowing for unbottlenecked IO when transferring image data.

One inference cycle starts by loading model weights from SPI Flash to an internal queue. As our model uses depthwise-separable convolutions with kernels of size 3×3 , the latency of loading weights is minimal compared to the compute. The loaded weights are stored in a 9-element LUT RAM buffer, where each compute unit has parallel access to every weight. We implement double buffering so that the next batch of weights is already in LUT RAM and is swapped via a MUX.

Each compute unit is a DSP block, connected to an input and output FIFO, surrounded by state management logic. Each compute unit’s FIFO is loaded with signal inputs corresponding to a series of kernel strides. To operate under BRAM constraints, each BRAM feeds a group of 4 compute units. This is possible because they operate in lockstep. Quantization parameters such as signal zero point, bias value, and effective output scale in terms of a shift value are loaded into the unit.

In depthwise mode, each compute unit produces an output after 9 Mac operations, and appropriate scaling. In pointwise model, we proceed in an output stationery fashion, keeping an output tile in BRAM. This tile is accumulated into, as each compute unit processes each channel. Our FIFO based approach allows us to mesh in with DDR burst reads, while accommodating for memory timing constraints and asynchronously running the DSPs at a higher clock.

Activations such as ReLU are fused with convolutional outputs, where each compute module adjusts for the ReLU layers quantization factors, and forwards a zero point value if the output is below the layers threshold. These outputs are written back into the DRAM. Similarly, our use of BatchNorm operations allows them to be fused in the scalepoint output.

The scheduling system is responsible for reading 3 scan lines in parallel from the memory interface and queuing them into the FIFO of each compute unit. It manages the write back

of the convolved signal from compute units, preserving order, and moving intermediate tensors back via the memory interface. The processed output is then shifted back to the MCU across the parallel bus to be post-processed and transmitted.

B. Background-Aware Bottleneck (BAB) Compression → Edge

Once the MiniEncoder produces a latent representation of the image, the second stage of edge processing begins: adaptive bitrate compression using BAB.

Conventional neural compression systems typically rely on heavy hyperprior networks to model the probability distributions of the latent representation in order to effectively entropy encode the latent. While effective, these networks add tens of millions of parameters and significant memory overhead, which is infeasible on constrained edge devices. BAB leverages the static nature of camera trap scenes to replace this heavy infrastructure with a lightweight, parameter-free module that exploits background redundancy for compression.

Methodology: The BAB’s methodology is centered on a simple yet highly effective principle: difference encoding against a pre-computed codebook. Each camera trap maintains a small latent codebook of representative background images in flash that capture the diversity of the deployment region (such as one captured during the day and one at night). During initial deployment, we run inference on the background images to produce background latents to create a codebook and store it on both edge and cloud. All subsequent images are then encoded into a quantized latent representation, which is first dequantized into, z_{fg} , and then compared against all background latents z_{bg} from the codebook. To select the representative latent, BAB simply calculates the L1 distance between the key vectors and chooses the latent with the smaller distance:

$$i^* = \arg \min_i \|z_{fg} - z_{bg,i}\|_1 \quad (3)$$

$$\Delta z_{i^*} = z_{fg} - z_{bg,i^*} \quad (4)$$

We then uniformly quantize the Δz with a step size of 1 to create $\hat{\Delta z}$, to construct a sparse latent where a small set of values occur with very high frequency.

$$\hat{\Delta z}_{i^*} = \lfloor \Delta z_{i^*} + 0.5 \rfloor \quad (5)$$

This low-entropy structure is highly compressible using z-lib, a lightweight entropy coder, eliminating the need for any learned entropy model.

Adaptive Bitrate: BAB leverages the structured latent space, wherein semantically similar images correspond to nearby latent representations. This property is exploited to enable adaptive bitrate control through a scalar parameter α , which scales the difference vector prior to quantization. A smaller α maps Δz to a narrower dynamic range with fewer quantization symbols, thereby achieving higher compression but increasing the deviation from the original foreground image due to amplified quantization error. After scaling, $\hat{\Delta z}$ is uniformly quantized and then entropy encoded via zlib. This

mechanism provides a single post-processing control knob to trade off bitrate and reconstruction quality without retraining or modifying the encoder and having any additional energy cost.

Prior to deployment, a predefined timer can be configured to periodically refresh the background reference. In such instances, the BAB pipeline is bypassed, and the MiniEncoder output is directly quantized through uniform quantization with a scale and zero-point. This incurs a higher transmission payload but ensures that the system remains adaptive to long-term environmental changes. Through background reuse, sparse difference encoding, and lightweight entropy coding, BAB replaces a $\sim 24\text{M}$ parameter hyperprior pipeline with a fully non-neural solution occupying only $\sim 800\text{K}$ INT8 parameters, reducing memory usage by over $120\times$ while maintaining competitive compression performance.

MCU Implementation: This lightweight algorithm is implemented on the ESP32 microcontroller, which manages the BAB workflow and final data transmission. Background latents are stored in flash as .bin files under FatFS. At runtime, the foreground latent is buffered in PSRAM, and a initial difference with each background candidate is computed in place by subtracting every tenth element. A comparison is done to see which of the two subtractions yielded the lower L1 distance, and then a full scale subtraction is done with the corresponding background candidate. Since L1 loss requires only accumulation, no additional storage beyond an accumulator is needed.

Because flash access is relatively slow, the latent is streamed in fixed-size chunks using a double-buffering scheme. While one buffer is processed, the other is refilled to maintain throughput. The two ESP32 cores are allocated unevenly: Core 0 executes approximately 70% of the compute, while Core 1 executes the remaining 30% and manages buffer refills. This streaming mechanism could be offloaded to DMA, but in our implementation, it is handled by Core 1 to avoid additional hardware complexity. All latent elements are represented in FP16, which strikes a balance between precision, memory footprint, and transfer bandwidth.

Once the closest background latent is identified, a full elementwise subtraction is performed in PSRAM by both the cores. The resulting difference vector is normalized, quantized, and packaged together with metadata including the selected reference index and the scaling factor, α .

Compression is performed using zlib in streaming mode to limit RAM overhead. A simple acknowledgment-based protocol is employed: if a packet is not acknowledged after two retries, it is skipped to preserve throughput. As uplink reliability is not the focus of this work, we rely on standard retransmission strategies.

C. BAB Decompression $\rightarrow$ Cloud

Once the compressed bitstream arrives from the edge device, the cloud-side pipeline takes over to perform the final reconstruction. The transmitted bitstream is first decoded using zlib, yielding the BAB latent representation, scaling factor,

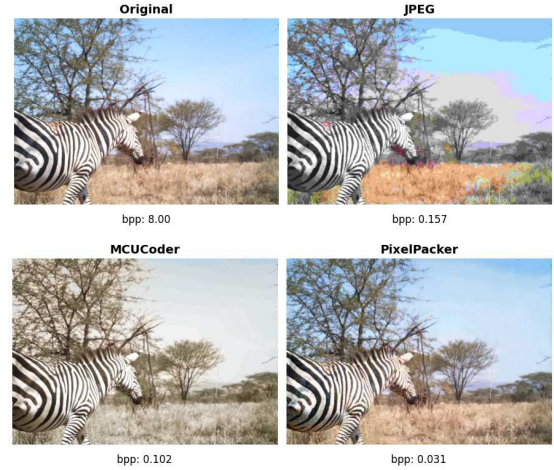


Figure 5: Visual Quality at Maximum Compression.

α , and the corresponding codebook index. This latent difference is first scaled and then combined with the appropriate background template (retrieved from a cloud-side codebook using the index) to restore the complete latent feature representation of the captured image:

$$\hat{z}_{fg} = z_{bg,i^*} + \frac{\hat{\Delta}z}{\alpha}. \quad (6)$$

D. BAB Fine-tuned Teacher Decoder $\rightarrow$ Cloud

With the full latent representation restored, the second module generates the final, high-fidelity image. This is handled by a powerful MS-ILLM decoder that has been specifically fine-tuned for our pipeline. The fine-tuning process is crucial for correcting artifacts introduced by the edge-side quantization and differencing steps. We fine-tune the decoder using the Real-world Object Removal Dataset (RORD), which provides a large collection of paired background and foreground images [46]. The image pairs are processed through the MiniEncoder and the BAB algorithm to generate BAB latents, which are then mapped to a modified latent representation $\hat{z}$. The decoder learns to transform $\hat{z}$ into the image space, producing $\hat{x}$, a refined reconstruction that more closely preserves perceptual fidelity to the original image. To prevent forgetting of the decoder’s pre-trained capabilities, we employ a small learning rate with a step scheduler. The training objective integrates multiple complementary loss functions of MSE and MS-SSIM.

V. EVALUATION

We evaluated PixelPacker against JPEG and MCUCoder. Our assessment focused on system performance, efficiency, and cost in the context of low-latency monitoring. In terms of performance, we targeted ultra-low bpp rates, where we demonstrate that PixelPacker maintains acceptable image quality while other methods experience significant degradation. For efficiency, we first evaluate the energy efficiency of our FPGA and compare it to other edge options. We also conducted a parallel comparison where other methods generate larger

payload sizes for comparable quality, which in turn leads to higher latency and energy consumption due to the transmission overhead in LoRaWAN settings.

A. System Performance

Image compression performance is determined by the rate-distortion trade-off. We measure compression performance using the bpp required to encode an image. For reconstruction fidelity, we employ Image Quality Assessment (IQA) metrics that focus solely on perceptual quality. Specifically, we use TReS (Transformer for Image Quality Assessment), PI (Perceptual Index), BRISQUE, and LPIPS. TReS, PI, and BRISQUE are reference-free, while LPIPS requires a reference image, ensuring both absolute and comparative evaluation of perceptual quality. We deliberately avoid PSNR, due to its bias toward smoother, blurrier reconstructions that penalize fine-grained details. [47]

Unlike other codecs, PixelPacker’s performance depends on the background redundancy that it leverages. Evaluating this requires Background and Region of Interest (BG/ROI) image pairs, which are absent in existing large-scale camera trap datasets. To address this, we curated a diverse testbed of 1,500 BG/ROI image pairs from the Serengeti dataset. For a subset of randomly selected camera locations, two false-trigger images (i.e., frames with no animals present) were manually chosen as representative backgrounds for day and night conditions respectively. Each background was paired with ROI-containing images captured at the same location to preserve scene context. False triggers that showed only minimal deviations from the background were excluded, as including them would artificially inflate performance estimates. The images in the testbed were manually annotated and selected for their diversity, featuring a wide spectrum of examples with varying ROI sizes and color variations, including instances with multiple disjoint ROIs and scenes exhibiting temporal and seasonal variations. Examples of such cases are illustrated in 6.

We evaluate PixelPacker’s rate-distortion performance on the curated testbed. As shown in Fig. 7, PixelPacker consistently achieves a superior tradeoff compared to JPEG and MCUCoder. It operates at bitrates as low as 0.01 bpp, corresponding to a compressed payload of only 7.8 kB for a 1080p image, while maintaining high perceptual quality with BRISQUE, TReS, and PI scores of 34.57, 41.99, and 4.8347, respectively. Notably, PixelPacker reaches compression levels beyond the minimum configurations of both baselines while preserving comparable perceptual quality. We show that PixelPacker attains an LPIPS of 0.5702 at just 0.023 bpp, compared to MCUCoder’s 0.094 bpp and JPEG’s 0.2318 bpp — representing approximately 4 $\times$ and 10 $\times$ higher compression, respectively. Qualitative results in Fig. 5 further demonstrate that while baseline codecs produce severely degraded outputs at comparable bitrates, PixelPacker preserves perceptual fidelity, which is required for downstream analysis.

To evaluate the performance of BAB, we conduct a two stage sensitivity analysis: i) to examine the effect of divergence between the reference background and ROI image pairs on

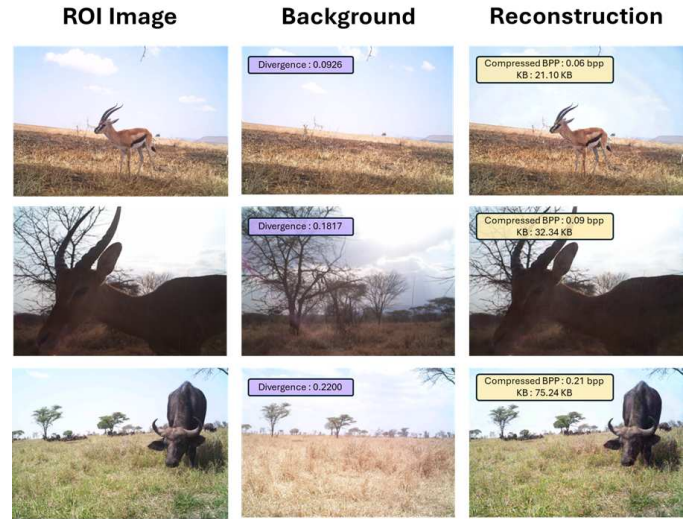


Figure 6: ROI Coverage Categories and their effect on compression rate at fixed $\alpha = 0.5$ at 1080p.

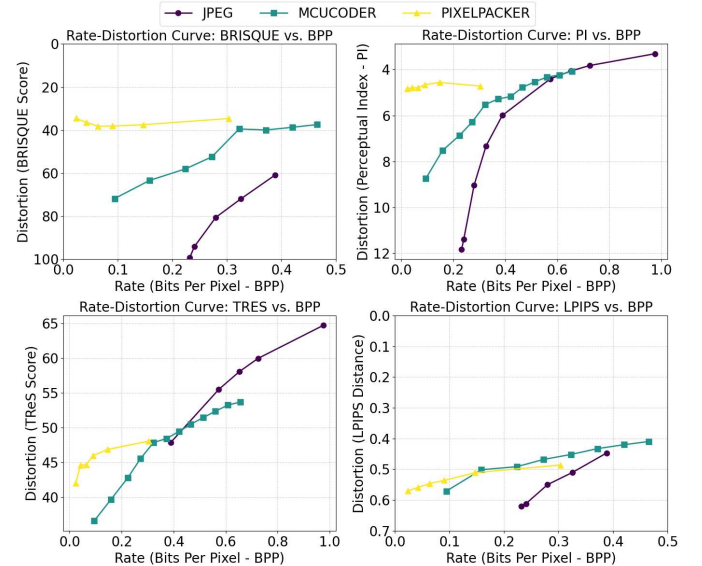


Figure 7: Rate-distortion curves comparing JPEG, MCUCoder, and PixelPacker across multiple perceptual quality metrics: BRISQUE, Perceptual Index (PI), TReS, and LPIPS. Lower distortion values indicate higher perceptual quality. PixelPacker consistently achieves lower distortion across metrics at substantially lower bpp.

compression and quality ii) effect of α on quality. To quantify this, we measure the L1 distance between the background and the ROI latents and calculate IQA metrics. The results are shown in Fig. 8, which exhibit a logarithmic-like increasing trend for bpp and LPIPS at varying levels of α . However, even under worst-case deviation, PixelPacker’s least compression setting ($\alpha = 0.25$) maintains a bitrate of just 0.03 bpp at an LPIPS of 0.65, remaining significantly better than both JPEG and MCUCoder. This demonstrates the robustness of the Background-Aware Bottleneck to realistic scene variability.

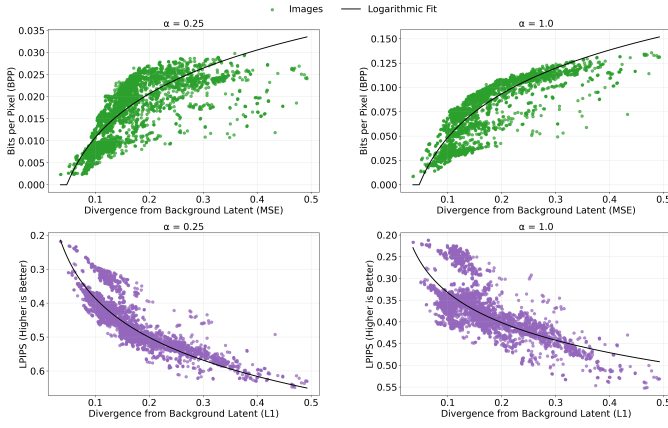
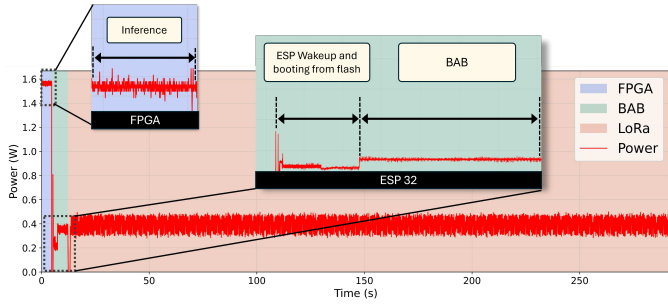
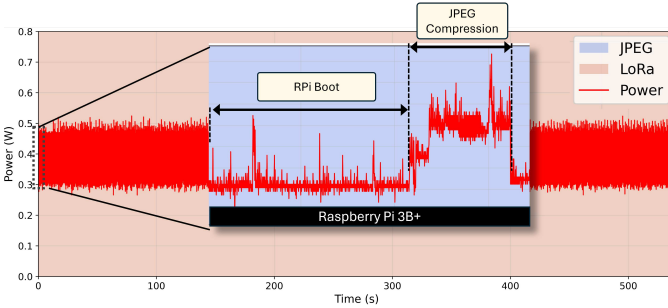


Figure 8: Sensitivity Analysis of BAB for increasing background latent divergence (lower is better).



(a) Our solution (PixelPacker): A short, high-power FPGA inference burst is followed by low-power processing on the ESP32, resulting in low overall latency.



(b) Baseline (JPEG on RPi): A long boot time is followed by a sustained, high-power period for JPEG compression, leading to high latency.

Figure 9: Power consumption profiles used for energy calculations. Our system (a) completes its tasks much faster than the baseline (b), significantly reducing transmission energy.

B. System Efficiency

PixelPacker’s feasibility in resource-constrained environments hinges on its power consumption, latency, and deployment cost, all of which are critically dependent on hardware selection. We assessed our system’s energy efficiency on an ESP32S3 with Xilinx Artix-7 FPGA coprocessor compared to a Raspberry Pi 3 Model B+ by measuring the total energy and latency for compressing a single image. The FPGA Co-Processor significantly outperforms the Raspberry Pi, compressing a 1080p image in 4.57 seconds, which is approximately 33 times faster than the Raspberry Pi’s 150.03 seconds.

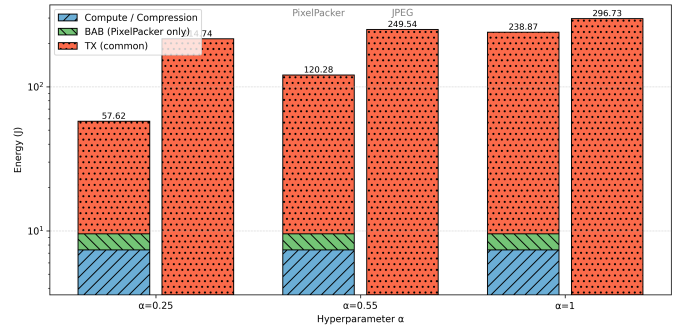


Figure 10: Decoupled Energy Consumption for JPEG and PixelPacker at a constant perceptual quality of LPIPS with transmission rate of 37kbps.

Crucially, this speed advantage is achieved with comparable energy efficiency: the FPGA consumed 1.81 J/Image during compression, nearly matching the Raspberry Pi’s 1.76 J/Image (Fig. 9a).

Another important consideration is the power consumption of PixelPacker when compared with JPEG and MCUCoder. While both of these would consume less power during compression, a complete analysis must account for the energy needed for both compression and transmission of the data. To make a fair comparison, we also ran all competing algorithms on the Raspberry Pi, except for PixelPacker, which we also assessed with the FPGA accelerator. For evaluating transmission energy, we selected LoRaWAN as the transmission network of choice, specifically the the RA-02 LoRaWAN variant. LoRaWAN is widely deployed in IoT scenarios with unreliable connectivity, and there exists a need for long-range transmission over several kilometers. However, the same analysis can be extended to other networks, such as satellite links, since the primary limiting factor, regardless of network type, is the available transmission bandwidth. For our analysis, we tested our pipeline for LoRa transmission at a transmission rate 0.3kbps, the standard maximum range with a spreading factor of 12.

As shown in 10, Pixel Packer on the ESP coupled with an FPGA achieves lower energy overhead at varying payload sizes compared to JPEG. This allows for a total latency of 121s, far lower than JPEG on Raspberry Pi’s latency of 539 s. Even though the FPGA’s instantaneous power draw is the highest, its extremely short transmission time proves more efficient than just JPEG on an SBC (Fig. 9b), which requires longer operational time to deliver a payload of the same size and quality.

VI. CONCLUSION

In this work, we presented PixelPacker, an end-to-end neural image compression framework tailored for wildlife monitoring under stringent edge constraints. Through an asymmetric edge-cloud design, PixelPacker achieves a balance between compression efficiency and reconstruction fidelity, overcoming the limitations of traditional codecs at low bitrates and neural models constrained by embedded power budgets. A hardware-efficient encoder implemented on FPGA and a BAB

module that removes redundant static content enable extreme compression with low latency and high energy efficiency. For future applications, PixelPacker can be adapted for other static-camera applications and can be extended to video, where temporal redundancies can be exploited too such as CCTV surveillance.

REFERENCES

- [1] J. Dertien, H. Negi, E. Dinerstein, K. Ramesh, S. Negi, R. Gopal, S. Gulick, K. Pathak, M. Kapoor, P. Yadav, M. Benitez, M. Ferreira, A. Wijnveen, A. Lee, B. Wright, R. Baldwin *et al.*, “Mitigating human-wildlife conflict and monitoring endangered tigers using a real-time camera-based alert system,” *BioScience*, vol. 73, pp. 1–10, 09 2023.
- [2] R. Sollmann *et al.*, “Risky business or simple solution – relative abundance indices from camera-trapping,” *Biological Conservation*, vol. 159, pp. 405–412, 2013.
- [3] M. A. Tabak *et al.*, “Machine learning to classify animal species in camera trap images: Applications in ecology,” *Methods in Ecology and Evolution*, vol. 10, no. 4, pp. 585–590, 2019.
- [4] J. A. Zwerts *et al.*, “Methods for wildlife monitoring in tropical forests: Comparing human observations, camera traps, and passive acoustic sensors,” *Conservation Science and Practice*, vol. 3, no. 12, p. e568, 2021.
- [5] S. S. Mason, R. A. Hill, M. J. Whittingham, J. Cokill, G. C. Smith *et al.*, “Camera trap distance sampling for terrestrial mammal population monitoring: lessons learnt from a uk case study,” *Remote Sensing in Ecology and Conservation*, vol. 8, no. 5, pp. 717–730, 2022.
- [6] C. Sun, C. Beirne, J. M. Burgar, T. Howey, J. T. Fisher *et al.*, “Simultaneous monitoring of vegetation dynamics and wildlife activity with camera traps to assess habitat change,” *Remote Sensing in Ecology and Conservation*, vol. 7, no. 4, pp. 666–684, 2021.
- [7] C. E. Jacobs *et al.*, “An evaluation of camera trap performance – what are we missing and does deployment height matter?” *Remote Sensing in Ecology and Conservation*, vol. 4, no. 4, pp. 352–360, 2018.
- [8] V. Demiroglou *et al.*, “Nod: A content retrieval scheme for intermittently-connected iot networks,” *Ad Hoc Networks*, vol. 130, p. 102825, 2022.
- [9] M. Zhang, “Drone-enabled internet-of-things relay for environmental monitoring in remote areas without public networks,” *IEEE Internet of Things Journal*, vol. 7, pp. 7648–7662, 08 2020.
- [10] R. D. Campbell *et al.*, “The use of remote cameras for monitoring cage traps during animal trapping,” *Ecological Research*, vol. 30, no. 5, pp. 963–967, 2015.
- [11] S. Newey, P. Davidson G, S. Nazir, G. Fairhurst, F. Verdicchio *et al.*, “Limitations of recreational camera traps for wildlife management and conservation research: A practitioner’s perspective,” *Ambio*, vol. 44, pp. 624–635, 11 2015.
- [12] M. Preti *et al.*, “Insect pest monitoring with camera-equipped traps: Strengths and limitations,” *Journal of Pest Science*, vol. 94, no. 2, pp. 203–217, 2021. [Online]. Available: <https://doi.org/10.1007/s10340-020-01309-4>
- [13] P. Glover-Kapfer, C. A. Soto-Navarro *et al.*, “Camera-trapping version 3.0: current constraints and future priorities for development,” *Remote Sensing in Ecology and Conservation*, vol. 5, no. 3, pp. 209–223, 2019.
- [14] C. Chalmers, P. Fergus, S. A. Wich, S. N. Longmore, N. D. Walsh, L. Oliver, J. Warrington, J. Quinlan, and K. Appleby, “Ai-driven real-time monitoring of ground-nesting birds: A case study on curlew detection using yolov10,” *ArXiv*, vol. abs/2411.15263, 2024.
- [15] S. Beery, G. Wu, V. Rathod, R. Votel, and J. Huang, “Context r-cnn: Long term temporal context for per-camera object detection,” in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 13 072–13 082.
- [16] S. E. Green, P. A. Stephens, M. J. Whittingham *et al.*, “Camera trapping with photos and videos: implications for ecology and citizen science,” *Remote Sensing in Ecology and Conservation*, vol. 9, no. 2, pp. 268–283, 2023.
- [17] S. Green, “Photo or video? which is best for recording mammals using camera traps?” <https://mammal.org.uk/blog/2022/10/photo-or-video-which-is-best-for-recording-mammals-using-camera-traps>, Oct 2022, accessed: 2025-10-02.
- [18] V. Bonilla *et al.*, “A systematic literature review of lorawan: Sensors and applications,” *Sensors*, vol. 23, no. 20, 2023.
- [19] K. Maine *et al.*, “Overview of iridium satellite network,” 12 1995, p. 483.
- [20] G. Wallace, “The jpeg still picture compression standard,” *IEEE Transactions on Consumer Electronics*, vol. 38, no. 1, pp. xviii–xxxiv, 1992.
- [21] F. Yang, L. Herranz, Y. Cheng *et al.*, “Slimmable compressive autoencoders for practical neural image compression,” in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 4996–5005.

- [22] A. Swanson *et al.*, “Snapshot serengeti, high-frequency annotated camera trap images of 40 mammalian species in an african savanna,” *Scientific Data*, vol. 2, p. 150026, 2015. [Online]. Available: <https://doi.org/10.1038/sdata.2015.26>
- [23] G. Ginesu *et al.*, “Objective assessment of the webp image coding algorithm,” *Signal Processing: Image Communication*, vol. 27, p. 867–874, 09 2012.
- [24] T. Cover *et al.*, *Elements of Information Theory*. Wiley, 2012. [Online]. Available: <https://books.google.com.pk/books?id=VWq5GG6ycxMC>
- [25] D. P. Kingma *et al.*, “Auto-encoding variational bayes,” 2013.
- [26] I. J. Goodfellow *et al.*, “Generative adversarial networks,” 2014.
- [27] S. Zhao *et al.*, “Towards deeper understanding of variational autoencoding models,” 2017.
- [28] Y. Blau *et al.*, “The perception-distortion tradeoff,” in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Jun. 2018, p. 6228–6237. [Online]. Available: <http://dx.doi.org/10.1109/CVPR.2018.00652>
- [29] J. Ballé *et al.*, “Variational image compression with a scale hyperprior,” 2018.
- [30] D. Minnen *et al.*, “Joint autoregressive and hierarchical priors for learned image compression,” 2018.
- [31] S. K. Aithal *et al.*, “Understanding hallucinations in diffusion models through mode interpolation,” 2024.
- [32] G. Hinton *et al.*, “Distilling the knowledge in a neural network,” 2015.
- [33] B. Jacob *et al.*, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” 2017.
- [34] —, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.
- [35] F. Chollet, “Xception: Deep learning with depthwise separable convolutions,” 2017.
- [36] A. Hojjat *et al.*, “McuCoder: Adaptive bitrate learned video compression for IoT devices,” 2024.
- [37] Y. Mao, J. Li *et al.*, “Easz: An agile transformer-based image compression framework for resource-constrained IoTs,” 2025.
- [38] A. Mazouz *et al.*, “Lightweight embedded FPGA deployment of learned image compression with knowledge distillation and hybrid quantization,” 2025.
- [39] S. Leorna *et al.*, “Estimating animal size or distance in camera trap images: Photogrammetry using the pinhole camera model,” *Methods in Ecology and Evolution*, vol. 13, no. 8, pp. 1707–1718, 2022.
- [40] G. S. Cheema *et al.*, *Automatic Detection and Recognition of Individuals in Patterned Species*. Springer International Publishing, 2017, p. 27–38.
- [41] M. J. Muckley *et al.*, “Improving statistical fidelity for neural image compression with implicit local likelihood models,” 2023. [Online]. Available: <https://arxiv.org/abs/2301.11189>
- [42] J. loup Gailly *et al.*, “zlib: A massively spiffy yet delicately unobtrusive compression library,” 2024, accessed: 2025-10-01.
- [43] W. Luo, Y. Li, R. Urtasun, and R. Zemel, “Understanding the effective receptive field in deep convolutional neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 29, 2016.
- [44] K. Simonyan *et al.*, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.
- [45] A. Kuznetsova *et al.*, “The open images dataset v4: Unified image classification, object detection, and visual relationship detection at scale,” *IJCV*, 2020.
- [46] M.-C. Sagong *et al.*, “Rord: A real-world object removal dataset,” in *British Machine Vision Conference*, 2022.
- [47] E. Agustsson *et al.*, “Multi-realism image compression with a conditional generator,” 2023.